

Reto Técnico Training QA

Voy a explicarles un poco las tecnologías que elegí y la forma en que la aplicación está construida en general. Como lenguaje de programación elegí la versión 21 de Java, ya que me parece que es una versión moderna y además es una versión LTS. La razón para escoger Java es que es un lenguaje maduro y robusto, con una gran comunidad y documentación extensa. El reto especificaba usar Programación Orientada a Objetos, y creo que no hay mejor lenguaje para eso que Java, ya que ese paradigma está en el corazón mismo del diseño del lenguaje. Además, mi mayor experiencia programando ha sido en Swift, así que la transición fue más sencilla que con otros lenguajes (como JavaScript, por ejemplo).

A Java lo estoy acompañando con el framework Spring con Spring Boot. El motivo para escoger dicho framework es porque también cuenta con mucho soporte en la comunidad y es casi que la opción de facto para crear soluciones backend en el ecosistema de Java. Y estoy usándolo con Spring Boot porque de esa forma se puede simplificar enormemente el desarrollo, ya que Spring Boot se encarga de gran parte de la configuración automáticamente, porque de otra forma tendríamos que configurar mil cosas, lo cual toma tiempo y muchos conocimientos.

Para la estructura del proyecto utilizamos Spring MVC, que implementa el patrón Modelo-Vista-Controlador solicitado en el reto. Esta implementación nos permite separar claramente la lógica de negocio, la presentación y el control de la aplicación. Y no se puede olvidar otro gran beneficio: al separar claramente las partes de una aplicación, es más sencillo luego realizar pruebas y hacer modificaciones sin tener que afectar todas las partes del programa o sin que otras partes afecten los resultados de nuestras pruebas.

En cuanto a la base de datos, elegí la base de datos H2, una base de datos relacional en memoria. La escogí porque es una opción ideal para el desarrollo y pruebas por su simplicidad y rapidez. Para interactuar con ella usé Spring Data JPA, ya que simplifica las operaciones de base de datos eliminando la necesidad de código SQL repetitivo.

Para la presentación del contenido, inicialmente pensé que lo más apropiado sería mostrar todo por consola, pero luego me di cuenta de que realmente esa no era la forma más adecuada para llevar a cabo de mejor manera los requerimientos de la prueba técnica. Así que al final usé Thymeleaf junto con HTML, CSS y un poquito de JavaScript. Esto agregó mucha más complejidad a la aplicación, pero creo que los resultados lo valen. Sirviéndome de la asistencia de la inteligencia artificial y de Google pude más o menos implementar algo funcional. Y como

Thymeleaf se integra naturalmente con Spring MVC y hace relativamente más sencillo crear interfaces web dinámicas y simples, no fue una mala elección.

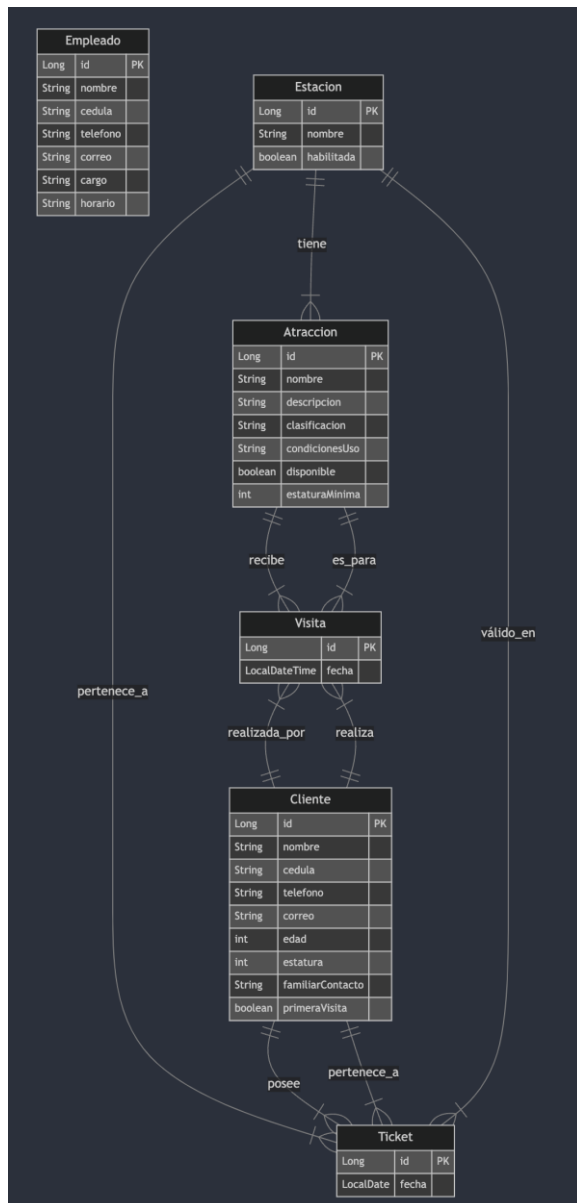
También incluí Lombok para reducir el código repetitivo, especialmente en la creación de getters, setters y constructores. Y ya para asegurar la calidad del código, estoy usando pruebas unitarias con JUnit 5 y Mockito, y de esta forma se puede verificar el funcionamiento correcto de cada componente.

También debo mencionar que la arquitectura del proyecto está organizada en tres capas principales: primero, la capa de presentación, que es la que maneja la interfaz de usuario y las interacciones. Segundo, la capa de servicio, la cual contiene toda la lógica de negocio. Y tercero y último, la capa de acceso a datos, cuya tarea es gestionar la persistencia y recuperación de información de la base datos. Y como mencioné antes, esta arquitectura en capas nos proporciona una separación clara de responsabilidades, algo que facilita el mantenimiento y la escalabilidad del proyecto (aunque este es un proyecto pequeño, siempre es bueno seguir buenas prácticas). Cada capa puede modificarse de manera independiente sin afectar a las demás, lo que hace el código más mantenible y flexible.

Por último, para implementar la funcionalidad del proyecto como tal he intentado siempre cumplir con lo pedido y he intentado implementar lo mejor posible las funcionalidades que se piden en los requerimientos. Solo he agregado unas cosas que tal vez no fueron pedidas explícitamente, pero que se asumen en el contexto. Por ejemplo, implementé mi propio sistema de descuentos, el cual envía promociones a clientes frecuentes basado en el número de visitas. Y sin ser más, voy a pasar a la parte del diagrama entidad relación.

Diagrama Entidad Relación

A continuación, se presenta el diagrama de la entidad relación y luego su explicación.



Y aquí una explicación simple del diagrama:

1. **Entidad Empleado:** Entidad aislada, sin relaciones con otras entidades.
2. **Entidad Estacion:**
 - Relacionada 1:N con **Atraccion** (una estación tiene muchas atracciones).
 - Relacionada 1:N con **Ticket** (un ticket está asociado con una estación).
3. **Entidad Cliente:**
 - Relacionada 1:N con **Visita** (un cliente puede realizar múltiples visitas).
 - Relacionada 1:N con **Ticket** (un cliente puede tener múltiples tickets).
4. **Entidad Atraccion:**

- Relacionada N:1 con **Estacion** (una atracción pertenece a una estación).
- Relacionada 1:N con **Visita** (una atracción puede ser visitada muchas veces).

5. **Entidad Visita:**

- Relacionada N:1 con **Cliente** (una visita pertenece a un cliente).
- Relacionada N:1 con **Atraccion** (una visita está asociada a una atracción).

6. **Entidad Ticket:**

- Relacionada N:1 con **Cliente** (un ticket es para un cliente).
- Relacionada N:1 con **Estacion** (un ticket corresponde a una estación).

Y esta es toda la información que quería brindarles. Les agradezco mucho por su valioso tiempo. ¡Les deseo un año nuevo feliz y lleno de éxitos!